Explore Munnar

Backend API Documentation & Postman Testing Guide

munnar-backend-dev · 3 features · 12 endpoints · 1 scheduled job

This document covers three backend features added in this development pass: the **Homepage Slider (Carousel) API**, the **Hotel Promoted / Verified flags**, and **automatic News Expiry**. Each section gives the endpoint contract (method, path, fields, responses) and a step-by-step Postman testing guide, including the exact test cases already verified against the live database during development.

Contents

01 Getting Started in Postman

All endpoints below live under the same Express app. Set up one Postman environment so every request can reuse a `{{baseUrl}}` variable instead of hardcoding a host.

Environment setup

1. Open Postman → click the **Environments** tab → **+** to create a new environment, e.g. "Munnar Backend — Local".
2. Add a variable named **`baseUrl`** with value `http://localhost:3000/api` for local testing (or your deployed backend URL + `/api`, e.g. `https://api.exploringmunnar.com/api`).
3. Select this environment from the dropdown in the top-right corner of Postman before running any request.
4. Every request path in this document is written relative to `{{baseUrl}}` — e.g. "POST /homepage-slides" means POST `{{baseUrl}}/homepage-slides`.

Response envelope

Every endpoint in this backend returns the same JSON envelope shape, whether it succeeds or fails. Check `success` first, then read `data` or `error` accordingly.

```
// Success
{
  "success": true,
  "code": 200,
  "message": "Human-readable description",
  "data": { /* endpoint-specific payload, or null */ },
  "timestamp": "2026-07-04T13:20:22.222Z"
}

// Error
{
  "success": false,
  "code": 400,
  "message": "Human-readable description",
  "error": { "type": "ValidationError", "details": "..." },
  "timestamp": "2026-07-04T13:20:22.222Z"
}
```

File uploads (multipart/form-data)

Any endpoint that accepts an image uses `multipart/form-data`, not raw JSON. In Postman: open the request → **Body** tab → select **form-data** → for each image field, change its type dropdown from "Text" to "**File**" and pick a file from disk. Do not set a Content-Type header manually — Postman sets the multipart boundary automatically.

> **IMPORTANT —** None of the endpoints documented here currently require an Authorization header. This matches the existing convention for admin routes in this codebase — worth revisiting before production launch, but it means you can test every request below with no auth setup.

02 Homepage Slider (Carousel) API

Lets an admin add and remove the hero images shown in the homepage carousel, replacing the previous static image. Slides are ordered by `position` (ascending), then creation time.

Method	Path	Purpose
POST	/homepage-slides	Upload a new slide (image + optional title/subtitle/position)
GET	/homepage-slides	List active slides, in display order
DELETE	/homepage-slides/:id	Remove a slide (deletes its Cloudinary image too)

POST `/homepage-slides`

Content-Type: multipart/form-data

Field	Type	Required	Description
`image`	file	Yes	The slide image (JPEG/PNG/WEBP). Uploaded to Cloudinary automatically.
`title`	text	No	Headline text overlaid on the slide, e.g. "Adventures & Cultural Treasures Await".
`subtitle`	text	No	Supporting line under the title.
`position`	text (number)	No	Sort order, ascending. Defaults to 0 if omitted.

Example success response — `201 Created`


```
{
  "success": true,
  "code": 201,
  "message": "Homepage slide created successfully",
  "data": {
    "id": "09ace6c0-2862-4522-a5ec-c4f54a1028bb",
    "imageUrl": "https://res.cloudinary.com/.../home-slides/qtf4wrpplr7yl4qgtfnt.png",
    "publicId": "home-slides/qtf4wrpplr7yl4qgtfnt",
    "title": "Adventures & Cultural Treasures Await",
    "subtitle": "Book cabs, bikes, or cars directly from Munnar's trusted drivers.",
    "position": 1,
    "is_active": true,
    "created_at": "2026-07-04T12:57:23.052Z",
    "updated_at": "2026-07-04T12:57:23.052Z"
  },
  "timestamp": "2026-07-04T12:57:23.517Z"
}
```


Error responses


```
// 400 — no image attached
{ "success": false, "code": 400, "message": "An image is required to create a slide",
  "error": { "type": "ApiError", "details": "An image is required to create a slide" } }
```

```
// 400 — an unrecognized field was sent (e.g. "caption" instead of "subtitle")
{ "success": false, "message": "property caption should not exist", "type": "ValidationError" }
```

Postman setup steps

- New request → **POST** `{{baseUrl}}/homepage-slides`.
- Body tab → **form-data**.
- Add key `image`, change type to **File**, choose an image.
- Add keys `title`, `subtitle`, `position` as Text (all optional).
- Send → expect **201** with the slide object shown above.

GET `/homepage-slides`

No parameters. Returns only active slides, ordered by `position` ascending.

```
{
  "success": true,
  "code": 200,
  "message": "Homepage slides fetched successfully",
  "data": [
    { "id": "...", "imageUrl": "...", "title": "...", "subtitle": "...", "position": 0, "is_active": true, ...
    { "id": "...", "imageUrl": "...", "title": "...", "subtitle": "...", "position": 1, "is_active": true, ...
  ],
  "timestamp": "2026-07-04T12:57:43.394Z"
}
```

DELETE `/homepage-slides/:id`

Deletes the Cloudinary image and marks the slide inactive. Deleting an already-deleted or unknown id returns `404 Slide not found`; deleting an already-inactive slide is idempotent and returns `200`.

```
// 200 — success
{ "success": true, "code": 200, "message": "Homepage slide deleted successfully", "data": null, ... }

// 404 — unknown id
{ "success": false, "code": 404, "message": "Slide not found",
  "error": { "type": "ApiError", "details": "Slide not found" } }
```

Postman test checklist (already verified against the live DB)

1. Create a slide with an image only (no title/subtitle/position) → 201, position defaults to 0.
2. Create a slide with no image attached → 400 "An image is required to create a slide".
3. Create a slide with an extra unexpected field → 400 validation error.
4. Create a second slide with position 1 → GET the list and confirm ordering matches position ascending.
5. DELETE the first slide by id → 200 → GET the list again and confirm it's gone.
6. DELETE the same id again → 200 (idempotent, not an error).
7. DELETE a random/nonexistent id → 404 "Slide not found".

03 Hotel — Promoted & Verified Flags

Two independent booleans on every hotel: **`isFeatured`** ("Promoted") controls sort order on the hotel listing page, and **`isVerified`** ("Verified") is a badge/trust indicator. Both can now be set directly when the admin creates the hotel — previously only `isVerified` worked at creation time; `isFeatured` was silently ignored and required a separate edit step. Both are also editable afterwards.

> **NOTE —** `isFeatured` = "Promoted", drives sort order. `isVerified` = "Verified", a trust badge only — it does not affect sort order.

Method	Path	Purpose
POST	/hotel	Create a hotel (accepts isFeatured & isVerified)
PUT	/hotel/:id	Edit a hotel (accepts isFeatured & isVerified)
PUT	/hotel/:id/promotions	Set homepage promo-card fields (separate from the two flags above)
POST	/hotels-list	List/filter hotels — promoted hotels always sort first
GET	/hotel/:id	Hotel detail — includes isFeatured & isVerified
DELETE	/hotel/:id	Soft-delete a hotel

POST `/hotel`

Content-Type: multipart/form-data

Field	Type	Required	Description
`name`	text	Yes	Hotel name.
`description`	text	No	Overview / about text.
`pricePerNight`	text (number)	Yes	Base nightly price in rupees.
`rating`	text (number)	No	e.g. 4.5
`distanceFromCenter`	text	No	e.g. "2km"
`stayType`	text	Yes	Category, e.g. "Eco Cottages".
`location`	text	Yes	Location name.
`locationRange`	text	Yes	One of the fixed area names (Munnar Town, Devikulam, ...).
`locationUrl`	text	No	Google Maps embed URL.
`amenities`	text (JSON string)	No	e.g. [{"title":"Room comfort","data":["Wifi"]}]

Field	Type	Required	Description
`experiences`	text (JSON string)	No	e.g. ["Campfire","Free WiFi"]
`rules`	text (JSON string)	No	e.g. [{"key":"checkin","title":"Check-in"}]
`isFeatured`	text ("true"/"false")	No	PROMOTED — NEW at creation. Shows this hotel first on the listing page.
`isVerified`	text ("true"/"false")	No	VERIFIED — shows a verified badge (already worked before this pass).
`images`	file[] (up to 10)	No	Field name must be exactly "images" for multiple files.

Example request (Postman form-data)

```
name:           Alpha Test Stay
description:    A quiet eco cottage near the tea plantations
pricePerNight:  3500
stayType:       Eco Cottages
location:       Munnar Town
locationRange:  Munnar Town
isFeatured:     true
isVerified:     true
images:         [file] hotel-photo.jpg
```

Example success response — `201 Created`


```
{
  "success": true,
  "code": 201,
  "message": "Hotel created successfully",
  "data": {
    "id": "e69bc1b6-5898-4903-8e66-908654bae9c6",
    "name": "Alpha Test Stay",
    "pricePerNight": "3500",
    "stayType": "Eco Cottages",
    "location": "Munnar Town",
    "isFeatured": true,
    "isVerified": true,
    "isActive": true,
    "isHighlighted": false,
    "isUniqueStays": false,
    "featuredImageUrl": null,
    "featuredUntil": null,
    "locationRange": "Munnar Town",
    "createdAt": "2026-07-04T13:12:59.468Z",
    "updatedAt": "2026-07-04T13:12:59.468Z"
  },
  "timestamp": "2026-07-04T13:13:00.582Z"
}
```

PUT /hotel/:id — editing the flags afterward

Same two fields, same values, sent as part of an edit. All fields are optional on edit — only send what changed. Also accepts `deletedImages` and `deletedRules` (JSON arrays of ids) to remove existing images/rules.

```
// PUT {{baseUrl}}/hotel/e69bc1b6-5898-4903-8e66-908654bae9c6   (form-data)
isFeatured: false
isVerified: true
```

PUT /hotel/:id/promotions — homepage promo card (separate feature)

Not part of this pass, but related: this existing endpoint controls the richer homepage "promotion card" (a separate highlighted image, an expiry date for the promotion, and `isHighlighted` / `isUniqueStays` flags). It does not affect the `isFeatured`/`isVerified` sort/badge behavior described above — the two systems are independent.

```
// PUT {{baseUrl}}/hotel/:id/promotions   (form-data)
isFeatured:     true      // required
isHighlighted:  true      // required
isUniqueStays:  false     // required
featuredUntil:  2026-08-01T00:00:00.000Z   // optional
featuredImage:  [file] promo-banner.jpg    // optional
```

POST /hotels-list — sort order proof

Filters and lists hotels. Promoted hotels (`isFeatured: true`) always appear before non-promoted ones, regardless of name — this is enforced server-side, not something the frontend needs to sort itself.

Field	Type	Required	Description
`pageNumber`	number	No	Defaults to 1. 10 hotels per page.
`stayType`	string[]	No	Filter by category.
`locationRange`	string[]	No	Filter by area.
`amenities`	string[]	No	Filter by amenity names.
`experiences`	string[]	No	Filter by experience names.
`budget`	{startingFrom, to}	No	Price range filter.
`isFeatured`	boolean	No	true = only currently-promoted hotels.
`isVerified`	boolean	No	true = only verified hotels.
`isHighlighted`	boolean	No	Promo-card filter (separate feature).
`isUniqueStays`	boolean	No	Promo-card filter (separate feature).
`isTopRated`	boolean	No	true = rating >= 3.5.

Example — verified sort-order test (as run during development)

```
// Request body
{ "pageNumber": 1, "stayType": ["Eco Cottages"], "locationRange": ["Munnar Town"] }
```

```
// Response — "Alpha" (promoted) ranks ABOVE "Aardvark" despite alphabetical order
{
  "success": true,
  "data": {
    "hotels": [
      { "name": "Alpha Test Stay",    "isFeatured": true  },
      { "name": "Aardvark Test Stay", "isFeatured": false },
      { "name": "Zebra Test Stay",    "isFeatured": false }
    ],
    "totalRecords": 3, "currentPage": 1, "totalPages": 1
  }
}
```

Postman test checklist (already verified against the live DB)

1. Create Hotel A with isFeatured=true and Hotel B with isFeatured=false, same stayType/locationRange, B's name alphabetically before A's.
2. POST /hotels-list filtered to that stayType/locationRange → confirm A (promoted) appears before B.
3. Create Hotel C with isVerified=true → GET /hotel/:id → confirm isVerified: true in the response.
4. PUT /hotel/:id with isVerified=false → GET again → confirm it flipped to false.
5. PUT /hotel/:id with isVerified=true → GET again → confirm it flipped back to true.
6. POST /hotels-list with isVerified: true → confirm only verified hotels are returned.

04 News Expiry (Automatic Deletion)

Admin sets an optional expiry date/time when creating a news item. A scheduled backend job checks every hour (and once immediately on server startup) for news past its expiry and **permanently deletes** it from the database — including its Cloudinary image. This is not an API call the frontend triggers; it runs automatically on the server.

Method	Path	Purpose
POST	/news	Create a news item (accepts expirationDate)
GET	/news	List news (optionally excluding expired items)
DELETE	/news/:id	Manually soft-delete a news item
—	(scheduled job)	Hourly: hard-deletes news whose expirationDate has passed

POST `/news`

Content-Type: multipart/form-data

Field	Type	Required	Description
`category`	text	Yes	Max 100 characters.
`heading`	text	Yes	Max 150 characters.
`detail`	text	Yes	Body text.
`image`	file	No	Uploaded to Cloudinary if present.
`expirationDate`	text (ISO datetime)	No	e.g. "2026-08-01T00:00:00.000Z". Omit for a news item that never expires.

Example success response — `201 Created`


```
{
  "success": true,
  "code": 201,
  "message": "News created successfully",
  "data": {
    "id": 35,
    "category": "Alert",
    "heading": "Road closure near Top Station",
    "detail": "...",
    "imageUrl": null,
    "isActive": true,
    "publicId": null,
    "expirationDate": "2026-08-01T00:00:00.000Z",
    "createdAt": "2026-07-04T13:36:35.965Z",
    "updatedAt": "2026-07-04T13:36:35.965Z"
  },
  "timestamp": "2026-07-04T13:36:36.409Z"
```

```
}
```

GET `/news`

Field	Type	Required	Description
`categories`	query string	No	Comma-separated, e.g. ?categories=Alert,Event
`search`	query string	No	Matches against the heading text.
`isNotExpired`	query string ("true")	No	Pass true to hide already-expired-but-not-yet-purged items.

IMPORTANT — The frontend should always pass `isNotExpired=true` on the public-facing news list, so expired items disappear from the site immediately — not just after the hourly cleanup job physically removes them.

DELETE `/news/:id`

Manual admin delete — this is a soft-delete (`isActive: false`), unlike the automatic expiry job which hard-deletes the row entirely. Use this for "remove now" from the admin dashboard; use expirationDate for "remove automatically on a future date."

How the automatic expiry job works

Behavior

1. Runs once immediately when the backend server starts (catches anything that expired while it was offline).
2. Then runs every hour on the hour.
3. Finds every news row where expirationDate is set and is in the past.
4. Deletes each one's Cloudinary image first (best-effort — a failed image delete does not block the row delete).
5. Hard-deletes the row from the database — it will not reappear, and it does not go through the /news/:id endpoint.

NOTE — Because this runs on a schedule inside the backend process, you cannot trigger the deletion itself from Postman. To verify it end-to-end: create a news item with expirationDate a few minutes in the past, then either wait for the next hourly run or restart the backend server (which runs the job immediately on boot), then GET /news again and confirm it's gone.

Postman test checklist (already verified against the live DB)

1. Create a news item with expirationDate set to yesterday.
2. Create a second news item with expirationDate set to next year.
3. Create a third news item with no expirationDate at all.
4. Trigger the cleanup (restart the server, or wait for the hourly run).
5. GET /news → confirm item 1 (expired) is gone, items 2 and 3 (future / no expiry) are still present.

05 Appendix — Reference

HTTP status codes used across these endpoints

Code	Meaning
200 OK	Read or delete succeeded.
201 Created	A new resource (slide / hotel / news item) was created.
400 Bad Request	Validation failed — missing required field, wrong type, or an unrecognized field.
404 Not Found	The id in the URL does not exist.
500 Internal Server Error	Unexpected server-side failure — check the backend logs.

Summary of every endpoint covered in this document

Method	Path	Feature
POST	/homepage-slides	Homepage Slider
GET	/homepage-slides	Homepage Slider
DELETE	/homepage-slides/:id	Homepage Slider
POST	/hotel	Hotel Promoted/Verified
PUT	/hotel/:id	Hotel Promoted/Verified
PUT	/hotel/:id/promotions	Hotel Promoted/Verified (related)
POST	/hotels-list	Hotel Promoted/Verified
GET	/hotel/:id	Hotel Promoted/Verified
DELETE	/hotel/:id	Hotel Promoted/Verified
POST	/news	News Expiry
GET	/news	News Expiry
DELETE	/news/:id	News Expiry

End of document. Generated for internal use while preparing the backend for frontend integration.